	Zewail City for Science and Technology 2025-2026

Course:	Numerical Method
Code:	MATH 307
type	Project

Due Date:	May 1, 2026
Project name:	Bilinear Interpolation for Image Rescaling

Under The Supervision of

Dr. Ahmed Abdelsamea

by:

Name	***ID***
Nour Eldin Mohamed	**202201310**
Mostafa Hisham	**202201998**

Table of Contents

1. Abstract

This project explores the application of numerical methods in image processing, specifically focusing on the rescaling and affine transformation of digital images. Because digital images are inherently represented as discrete grids of pixels, operations such as resizing or rotation often map coordinates to non-integer locations. Therefore, exact analytical solutions are of limited practical value. Instead, this project relies on numerical multidimensional interpolation techniques—specifically bilinear interpolation—to estimate missing pixel intensity values. The project utilizes Python to simulate the bilinear interpolation algorithm, calculating new sub-pixel intensities by combining the values of their four nearest neighbors, and compares the visual smoothness against basic nearest-neighbor duplication approaches.

2. Introduction

Digital images are fundamentally discrete entities, composed of a finite grid of picture elements—pixels—each assigned a specific intensity or color value at integer coordinate positions. This discrete nature, while well-suited for storage and display on digital hardware, poses a significant challenge when performing geometric transformations such as rescaling, rotation, or shearing (Gonzalez & Woods, 2018).

Image scaling, one of the most common operations in digital image processing, involves mapping each pixel in a destination (output) image back to a corresponding location in the source (input) image. When an image is enlarged or reduced, this mapping almost invariably produces non-integer coordinate values—points that fall between the defined pixel positions of the original discrete grid. For example, scaling a 100×100 pixel image up to 200×200 requires computing intensity values at coordinates such as (0.5, 0.5), (0.5, 1.5), and so on, which do not correspond to any actual pixel in the original image.

Because digital images only define pixel intensities at integer coordinates, it is mathematically impossible to retrieve an exact value at a fractional location. This is where numerical interpolation becomes indispensable. Interpolation provides a principled method for estimating the value of a function at a point that lies between known data points, thereby enabling the reconstruction of a continuous signal from its discrete samples. As noted by Chapra & Canale (2021), interpolation methods for one-dimensional

problems can be systematically extended to multidimensional cases through separable application of the same linear schemes.

Among the various interpolation strategies available, bilinear interpolation stands out as the standard numerical scheme for sub-pixel estimation in two-dimensional image processing. As stated in Sayed AbdelSamea (2026), the simplest case of 2D interpolation in Cartesian coordinates is precisely this bilinear approach. Unlike the simpler nearest-neighbor method—which assigns to each new pixel the intensity of the closest known pixel, resulting in a blocky and pixelated appearance—bilinear interpolation computes a weighted average of the four nearest pixels surrounding the target location. Ullah et al. (2025) confirm through comparative analysis that bilinear interpolation consistently outperforms nearest-neighbor methods in terms of image quality across a wide range of upscaling dimensions.

This report details the mathematical formulation of bilinear interpolation as covered in the course lecture notes, the computational methodology employed to implement it in Python, and an analysis of the results produced when compared against the nearest-neighbor approach.

3. Problem Definition

3.1 Formal Setup

Consider an original source image defined on a regular integer grid. When this image is subjected to an affine transformation—such as upscaling by a factor of 2—each pixel in the new destination image must be assigned an intensity value. Through the inverse affine mapping, the corresponding location in the source image is computed, yielding a point with fractional (sub-pixel) coordinates (x_i, y_i).

As formalized in the course lecture notes on spline and multidimensional interpolation (Sayed AbdelSamea, 2026), 2D interpolation deals with determining intermediate values for functions of two variables, $z = f(x_i, y_i)$. To determine the intensity at this sub-pixel location, bilinear interpolation utilizes the four nearest integer-grid neighbors that surround the point. These four known data points are:

- $f(x_1, y_1)$ — intensity at the top-left neighbor
- $f(x_2, y_1)$ — intensity at the top-right neighbor
- $f(x_1, y_2)$ — intensity at the bottom-left neighbor

- $f(x_2, y_2)$ — intensity at the bottom-right neighbor

where x_1, x_2 are the integer column coordinates immediately to the left and right of x_i, and y_1, y_2 are the integer row coordinates immediately above and below y_i.

3.2 Variable Definitions

Variable	Definition
x_i, y_i	Fractional (sub-pixel) spatial coordinates of the target point in the source image
x_1, x_2	Left and right bounding integer column coordinates ($x_1 = \lfloor x_i \rfloor$, $x_2 = x_1 + 1$)
y_1, y_2	Top and bottom bounding integer row coordinates ($y_1 = \lfloor y_i \rfloor$, $y_2 = y_1 + 1$)
$z = f(x_i, y_i)$	Unknown interpolated pixel intensity at the sub-pixel target location
$f(x_1, y_1)$, *etc.*	Known pixel intensities at the four surrounding integer grid positions
dx, dy	Fractional distances: $dx = x_i - x_1$, $dy = y_i - y_1$ (values $\in [0, 1)$)

3.3 Mathematical Formulation (Lagrange Form)

Bilinear interpolation proceeds in two sequential stages of one-dimensional linear interpolation, expressed using the Lagrange basis form, as presented in Sayed AbdelSamea (2026) and Chapra & Canale (2021).

Stage 1 — Interpolation along the x-direction (holding y fixed):

For the top row (at $y = y_1$):

$$f(x_i, y_1) = \cdot\,[(x_i - x_2) / (x_1 - x_2)] \cdot f(x_1, y_1) \;+\; [(x_i - x_1) / (x_2 - x_1)] \cdot f(x_2, y_1)$$

For the bottom row (at $y = y_2$):

$$f(x_i, y_2) = \cdot\,[(x_i - x_2) / (x_1 - x_2)] \cdot f(x_1, y_2) \;+\; [(x_i - x_1) / (x_2 - x_1)] \cdot f(x_2, y_2)$$

Stage 2 — Interpolation along the y-direction (using the two intermediate results):

$$f(x_i, y_i) = [(y_i - y_2) / (y_1 - y_2)] \cdot f(x_i, y_1) \;+\; [(y_i - y_1) / (y_2 - y_1)] \cdot f(x_i, y_2)$$

Substituting both stage-1 results into the stage-2 formula yields the fully expanded bilinear expression (Sayed AbdelSamea, 2026):

$$f(x_i,y_i) = [(x_i - x_2)(y_i - y_2) / (x_1 - x_2)(y_1 - y_2)]\, f(x_1,y_1) + [(x_i - x_1)(y_i - y_2) / (x_2 - x_1)(y_1 - y_2)]\, f(x_2,y_1)$$

$$+ [(x_i - x_2)(y_i - y_1) / (x_1 - x_2)(y_2 - y_1)]\, f(x_1, y_2) + [(x_i - x_1)(y_i - y_1) / (x_2 - x_1)(y_2 - y_1)]\, f(x_2, y_2)$$

Since $x_2 = x_1 + 1$ and $y_2 = y_1 + 1$, and letting $dx = x_i - x_1$ and $dy = y_i - y_1$ (each in [0, 1)), the formula simplifies to the compact area-weighted form:

$$f(x_i, y_i) = (1-dx)(1-dy) \cdot f(x_1, y_1) + dx(1-dy) \cdot f(x_2, y_1) + (1-dx)dy \cdot f(x_1, y_2) + dx \cdot dy \cdot f(x_2, y_2)$$

This compact expression makes clear that the interpolated value is a weighted sum of the four neighboring pixels, where the weights are proportional to the rectangular areas formed by the sub-pixel point and its four neighbors—hence the term "bilinear."

3.4 Worked Numerical Example from Course Material

As demonstrated in the lecture notes (Sayed AbdelSamea, 2026), consider measured temperatures at four corners of a rectangular heated plate:

Coordinate	Temperature T
T(2, 1) = 60	T(9, 1) = 57.5
T(2, 6) = 55	T(9, 6) = 70

To estimate the temperature at $x_i = 5.25$ and $y_i = 4.8$, substituting into the bilinear formula gives:

$$f(5.25, 4.8) = [(5.25-9)(4.8-6) / (2-9)(1-6)] \cdot 60 + [(5.25-2)(4.8-6) / (9-2)(1-6)] \cdot 57.5$$
$$+ [(5.25-9)(4.8-1) / (2-9)(6-1)] \cdot 55 + [(5.25-2)(4.8-1) / (9-2)(6-1)] \cdot 70 = 61.2143$$

This example, drawn directly from the course lecture notes, illustrates how bilinear interpolation naturally handles real-valued spatial coordinates in a physically meaningful setting.

4. Methodology

4.1 Context: From 1D to Multidimensional Interpolation

The theoretical foundation for bilinear interpolation rests on the same Lagrange linear interpolation framework covered extensively in the course lecture notes on spline interpolation (Sayed AbdelSamea, 2026). Just as 1D linear splines connect pairs of data points with straight lines using the slope formula, bilinear interpolation extends this idea into two spatial dimensions by applying the linear Lagrange formula first along x, then along y. This separability is precisely what makes the method computationally

efficient: a 2D problem is decomposed into two successive 1D problems, each requiring only two known values.

This connection to spline theory is significant. The numerical integration lecture notes (Sayed AbdelSamea, 2026) discuss how piecewise polynomial methods can be applied to functions available only at discrete tabulated points—exactly the situation encountered with digital images, where pixel intensities are known only at integer grid locations. Bilinear interpolation can thus be understood as a piecewise bilinear surface fitted between each 2×2 cell of pixels in the source image.

4.2 Inverse Affine Mapping

The computational pipeline for image rescaling using bilinear interpolation begins not in the source image, but in the destination image. For each pixel position (u, v) in the output image, the inverse affine transformation is applied to find the corresponding location (x_i, y_i) in the source image. This backward mapping approach is preferred because it guarantees that every pixel in the destination image receives a value, avoiding holes that can arise from forward mapping.

For a simple scaling transformation by factors (s_x, s_y):

$$x_i = u / s_x \quad and \quad y_i = v / s_y$$

The resulting coordinates (x_i, y_i) are generally non-integer (fractional), lying between the integer grid points of the source image.

4.3 Isolating Integer Coordinates and Fractional Distances

Once the fractional source coordinates are known, the algorithm determines the four surrounding integer neighbors:

- $x_1 = \lfloor x_i \rfloor$ (floor), $x_2 = x_1 + 1$
- $y_1 = \lfloor y_i \rfloor$ (floor), $y_2 = y_1 + 1$
- $dx = x_i - x_1$ (horizontal fractional distance, $\in [0, 1)$)
- $dy = y_i - y_1$ (vertical fractional distance, $\in [0, 1)$)

Boundary checks ensure that x_2 and y_2 do not exceed the image dimensions, clamping them to the last valid pixel row or column when necessary. This prevents index-out-of-bounds errors at image edges.

4.4 Applying the Bilinear Interpolation Formula

With the four neighbor intensities and the fractional distances in hand, the bilinear formula is applied to produce the interpolated pixel value. For a grayscale image, this yields a single scalar; for a color (RGB) image, the operation is applied independently to each of the three color channels, ensuring consistent color blending. The final interpolated value is then rounded to the nearest integer (for 8-bit images, clamped to [0, 255]) and written to the destination image array at position (u, v). This process is repeated for every pixel in the destination image, constructing the full rescaled output.

4.5 Software Tools and Libraries

Library	Role in the Simulation
Python	Core programming language for implementing the interpolation algorithm and control flow
NumPy	Efficient multi-dimensional array handling, floor operations, and vectorized pixel arithmetic
OpenCV (cv2)	Reading source images from disk, writing output images, and BGR ↔ RGB color-space conversions
Matplotlib (plt)	Side-by-side visualization of the original, nearest-neighbor, and bilinear-interpolated images

5. Results

5.1 Test Case: 2×2 to 8×8 Upscaling

To illustrate the behavior of the two interpolation approaches, consider a hypothetical minimal test case using a 2×2 pixel input image with the following intensity values:

0 (black)	255 (white)
128 (gray)	0/0/128 (navy)

This image is upscaled by a factor of 4, producing an 8×8 output. The two interpolation methods are applied and their outputs are compared.

5.2 Nearest-Neighbor Result

The nearest-neighbor method resolves each destination pixel by selecting the single closest source pixel—rounding the inverse-mapped coordinates to the nearest integer. Applied to the 2×2 → 8×8 upscaling, this produces four uniformly colored 4×4 quadrants with sharp, step-like boundaries between them. The resulting image has a distinctly blocky, pixelated appearance with no smooth transition whatsoever between the black, white, gray, and navy regions. This visual artifact—known as the "blocky" or "box" effect—becomes increasingly pronounced as the upscaling factor increases. As noted by Ullah et al. (2025), nearest-neighbor interpolation can also exhibit a considerable increase in execution time for larger images relative to its theoretical simplicity.

5.3 Bilinear Interpolation Result

In sharp contrast, the bilinear interpolation algorithm computes a weighted average of the four nearest neighbors at each destination pixel, proportional to the sub-pixel distances as established in the formulas of Section 3. For the same 2×2 → 8×8 upscaling, the output image displays smooth, continuous color gradients across the entire canvas. Near the center of the image, where all four corner pixels contribute, the intensities blend gradually—producing intermediate gray tones that transition naturally from black to white horizontally and from gray to navy vertically.

The visual quality improvement observed here aligns with the findings of Bilal et al. (2024), who demonstrated that bilinear interpolation serves as an effective foundation for image rescaling even in more advanced compression pipelines. Furthermore, as ScienceDirect Topics on image interpolation notes, bilinear interpolation estimates appropriate intensity pixel values by finding the distance-weighted average of the four surrounding neighbors, making it substantially superior to nearest-neighbor assignment for smooth natural images.

The key visual differences are summarized in the comparison table below:

Property	Nearest-Neighbor	Bilinear Interpolation
Edge appearance	Sharp, blocky step transitions	Smooth, gradual gradients
Computation cost	Very low (1 neighbor lookup)	Low (4 neighbor lookups + arithmetic)
Color blending	None — single pixel copied	Weighted blend of 4 neighbors

Artifact type	Pixelation / block effect	Slight blurring at sharp boundaries
Suitability	Fast previews, pixel art	Standard image resizing, photography

5.4 Discussion

The results confirm that bilinear interpolation is a substantially superior method for image upscaling when visual quality is paramount. By treating pixel intensity as a continuous, piecewise-bilinear function of spatial position and estimating sub-pixel values through weighted combinations of neighboring data points, it effectively reconstructs a smooth approximation of the underlying continuous image signal from its discrete samples.

The slight blurring introduced by bilinear interpolation at locations with originally sharp intensity transitions is a well-known trade-off: smoothness is achieved by averaging, which inherently attenuates high-frequency content. For applications requiring sharper edges, higher-order methods such as bicubic interpolation or Lanczos resampling may be preferred (Gonzalez & Woods, 2018), though at greater computational cost.

Nonetheless, bilinear interpolation represents an elegant and efficient compromise between computational simplicity and output quality. Its direct grounding in the Lagrange linear interpolation framework—which connects it to the broader family of Newton-Cotes and spline methods studied throughout the course (Sayed AbdelSamea, 2026; Chapra & Canale, 2021)—places it firmly within the domain of numerical analysis while remaining highly practical for real-world image processing applications.

6. Appendances

Github Repository: https://github.com/Noureqo1/Numerical_Bilinear_Interpolation-_Image_Rescaling

7. References

1. Mathematical Methods Applied to Digital Image Processing, accessed on May 1, 2026, https://digitalcommons.odu.edu/cgi/viewcontent.cgi?article=1074&context=ece_fa c_pubs

2. (PDF) Mathematical Methods Applied to Digital Image Processing - ResearchGate, accessed on May 1, 2026, https://www.researchgate.net/publication/275068907_Mathematical_Methods_Applied_to_Digital_Image_Processing

3. Numerical Methods in Image Processing | PDF | Partial Differential Equation - Scribd, accessed on May 1, 2026, https://www.scribd.com/document/393095602/Numerical-Methods-Image-Processi ng

4. A Step-by-Step Guide to Speech Recognition and Audio Signal Processing in Python, accessed on May 1, 2026, https://towardsdatascience.com/a-step-by-step-guide-to-speech-recognition-and-audio-signal-processing-in-python-136e37236c24/

5.Bilal, M., Ullah, Z., Mujahid, O., & Fouzder, T. (2024). Fast Linde–Buzo–Gray (FLBG) algorithm for image compression through rescaling using bilinear interpolation. Journal of Imaging, 10(5), 124. https://doi.org/10.3390/jimaging10050124

6.Mulla, R. (n.d.). Image processing with OpenCV and Python [Video]. YouTube.

7.Sayed AbdelSamea, A. (2026). Numerical analysis: Curve fitting — spline interpolation [Lecture notes]. Zewail City of Science, Technology and Innovation.

8.Sayed AbdelSamea, A. (2026). Numerical analysis: Numerical integration [Lecture notes]. Zewail City of Science, Technology and Innovation.